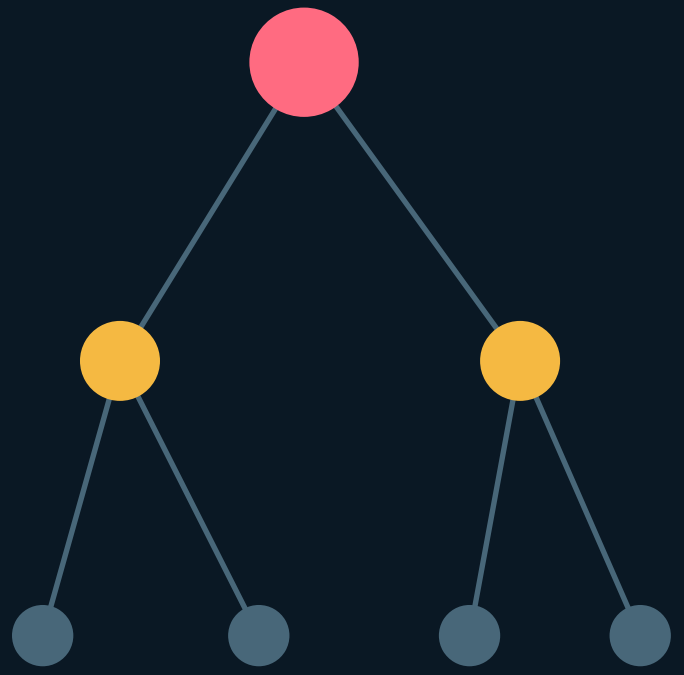
Predictive Modeling of Cloud System Failures



Map. Simulate. Score.

Dependency graphs + counterfactual risk estimation

Akhil Joshi

01 Symptoms
02 Problem
03 Map, Simulate, Score
04 Limitations
05 Real-world Comparison
06 Takeaways

Akhil Joshi

Cloud Platform Engineer at Koch Industries

Big Four alumnus: Deloitte + PwC

M.S. in Computer Science / Kansas State University

8+ peer-reviewed research papers

AWS + Azure certified

Cloud-native / CNCF community participant

OFF THE CLOCK

Marvel comics / Chess / Badminton

One incident. Three perspectives.

The customer A purchase or update does not complete.

The engineer Several true alerts; limited context.

The system Shared dependencies and delayed consequences.

AWS: the DNS fix was not the end.

00:00 / onset	DynamoDB endpoint resolution fails.
+00:50 / diagnosis	DNS state identified as the source.
+02:52 / DDB API recovery	Dependent systems still need recovery.
+14:32 / summary window ends	EC2 / networking recovery followed different paths.

Cloudflare: one bad file, hours of recovery.

11:20 / reported onset

Proxy failures and widespread HTTP errors.

Initial investigation

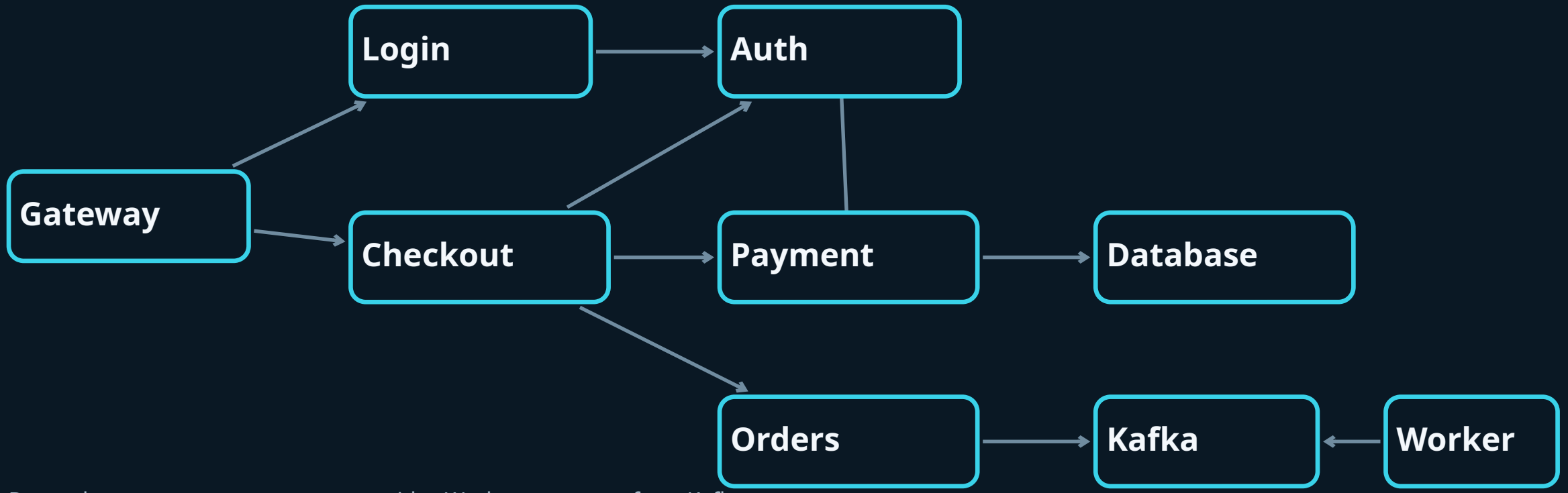
DDoS suspected; actual cause was configuration data.

14:30 / core traffic restored

About 3h 10m from the opening timestamp.

17:06: all systems restored / 5h 46m overall.

Shrink the same problem into one checkout journey.



Dependency arrows: consumer -> provider. Worker consumes from Kafka.

02:14 AM. The on-call alerts start arriving.

02:14 Gateway / 5xx errors

02:15 Login / slow responses

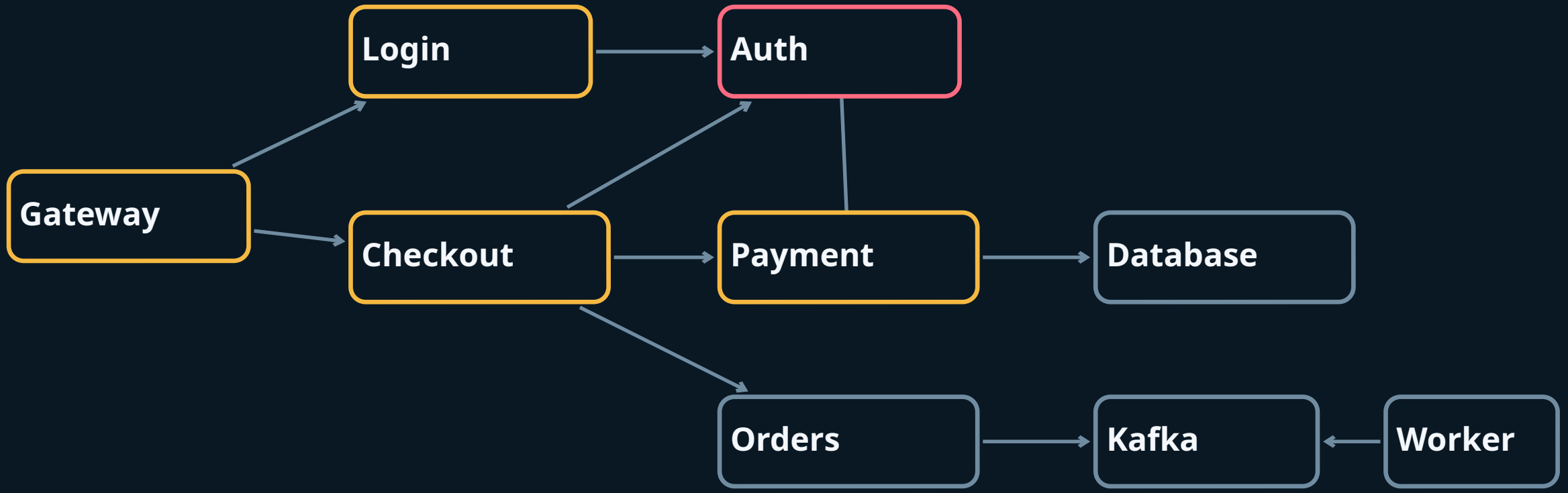
02:15 Checkout / p95 latency

02:16 Payment / timeouts

02:17 Checkout / retries rising

02:19 Journey / success rate falling

A shared dependency is a lead, not a verdict.



Auth could explain more symptoms. Verify it with traces and timing.

The problem is a missing system-level view.

Which component failures could create widespread impact?

Inputs: components + relationships + operating signals.

What would a useful answer give us?

Impact

Which customer journey could this failure affect?

Priority

Which scenarios deserve our attention first?

Evidence

What should we check, and why?

A better engineering decision, not an exact outage date.

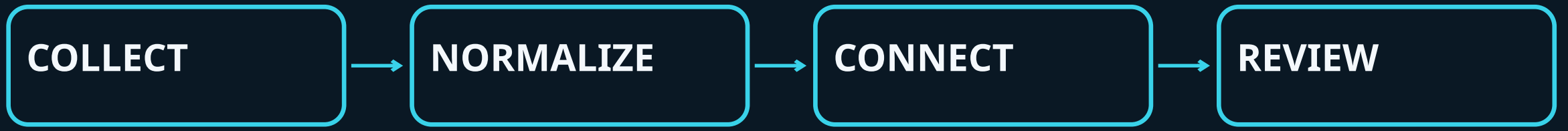
Start with an inventory, not a perfect diagram.

COMPONENT RECORD

Component: Payment service
Depends on: Auth + payment database
Relationship: Required API call / database connection
Context: Owner + environment + evidence + last verified

List the components first. Add the relationships to create a graph.

Generate relationships from evidence.



Inventory, configuration and traces.

Match component identities.

Typed, directed relationships.

Verify critical edges and gaps.

Keep provenance and freshness. Do not erase a known edge because its error correlation is low.

Topology gives paths. Telemetry adds context.

PATH

Who depends on whom?

Configuration + traces

STATE

What is changing now?

Latency + retries + saturation

BEHAVIOUR

What happens next?

Timeouts + fallbacks + queue lag

Signals inform scenario rules; correlation alone does not prove a dependency.

What is manual, automated or AI-assisted?

Software Collect, reconcile, traverse, repeat and calculate.

Engineers Define scope, failure behavior and impact limits.

AI / optional Suggest candidates, retrieve evidence and explain.

Simulate in the model, not in production.

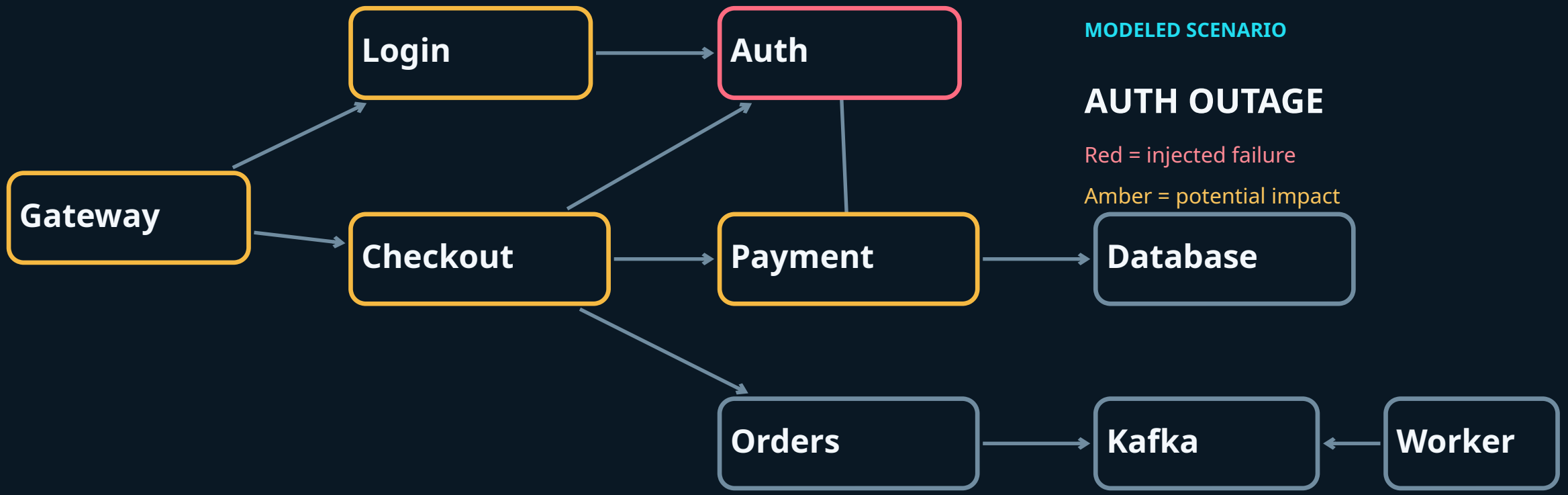
01 / Select + inject Choose one component and a defined failure mode.

02 / Follow + apply Traverse dependents using explicit impact rules.

03 / Measure + reset Record impact; restore the model for the next run.

04 / Repeat Try every component under varied conditions.

What if Auth fails?



MODELED SCENARIO

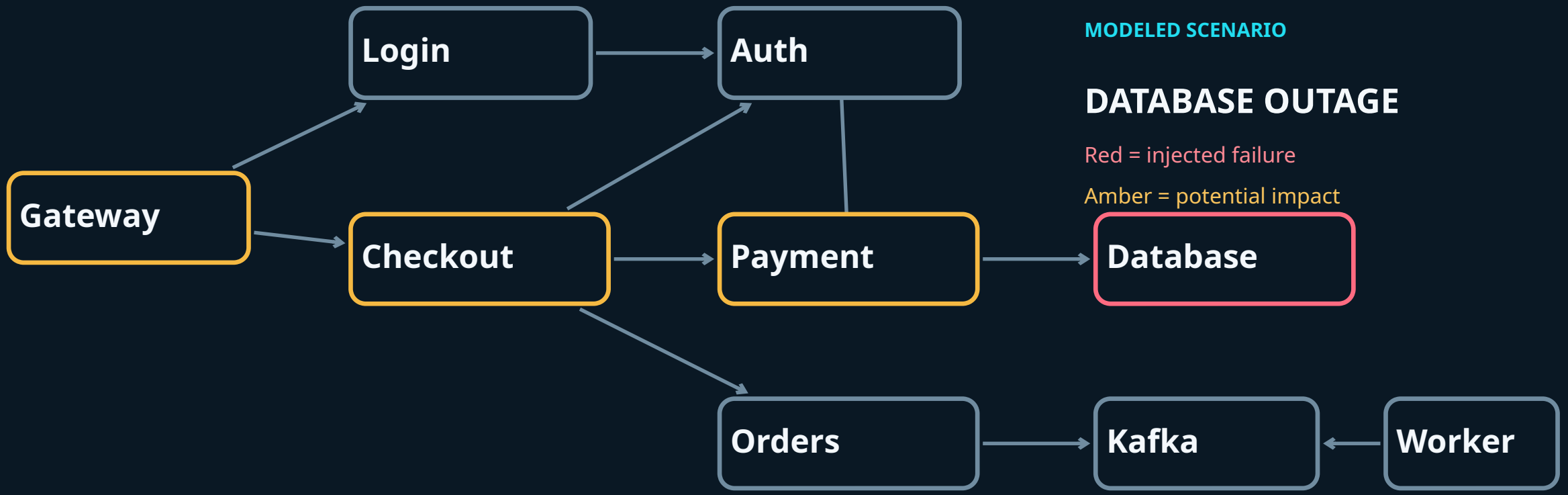
AUTH OUTAGE

Red = injected failure

Amber = potential impact

Potential: Login, Checkout, Payment and Gateway. Async infrastructure is outside this failure path.

What if Database fails?



MODELED SCENARIO

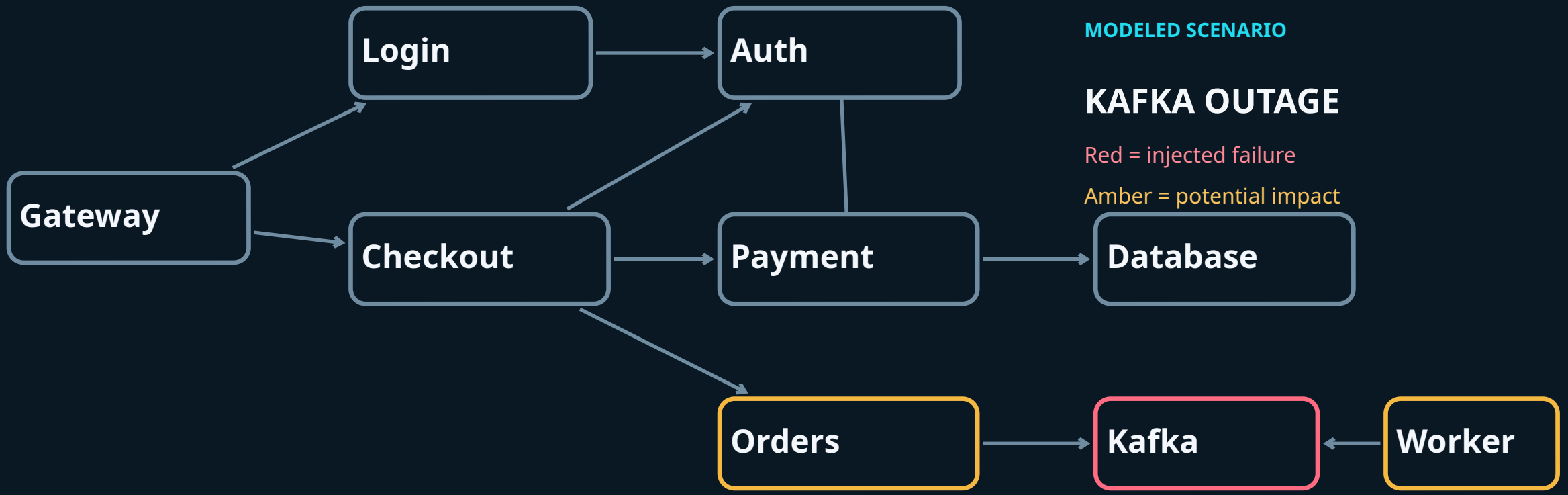
DATABASE OUTAGE

Red = injected failure

Amber = potential impact

Potential: Payment, Checkout and Gateway. Login is outside this database failure path.

What if Kafka fails?



MODELED SCENARIO

KAFKA OUTAGE

Red = injected failure

Amber = potential impact

Potential: Orders and Worker progress. Checkout impact depends on publication policy.

One equation. No memorisation required.

Risk = P(impact >= limit | selected failure)

How often does simulated impact cross our chosen line?

Research framing: DG-CFRM / Akhil Joshi + Akshitha Segireddy

Use the framing. Define an impact measure that matches your own system.

Define impact before calculating risk.

Teaching impact = degraded components / 9

Chosen line: 30% -> 3 or more components

Your team can choose customer impact, freshness or weighted journeys instead.

The simulator does the counting.

Estimated risk = runs crossing the limit / total runs

981 / 1000 = 0.981

Auth result from the included teaching simulation. Not a production probability.

Rank the modeled impact. Keep criticality separate.



Conditional impact only. A low cascade score is not low business criticality.

What our model cannot do.

Four limits worth saying out loud

Predict the exact time

It cannot tell us when a component will fail.

Prove the root cause

The exact migration, bug or config change needs evidence.

Fill in missing dependencies

Bad edges or stale telemetry can mislead the ranking.

Guarantee accuracy

A high scenario score is not a proven outage probability.

The score estimates impact conditional on an injected failure, not the chance of that failure occurring.

A missing edge is uncertainty, not independence.

Missing or stale edge	Keep evidence age; verify and refresh critical paths.
Shared infrastructure	Add node pools, zones, DNS and capacity constraints.
Weak evidence	Expose uncertainty. Do not invent dependencies.

Freshness, detail and compute: choose the trade-off.

More detail — More nodes, failure modes and trials mean more work.

Fresher graphs — Refresh after changes; show the age of the last result.

Practical scope — Start with critical journeys; rerun affected paths first.

Paper reports ~8-12 min for 1,000 nodes; not a universal runtime guarantee.

Test the model before trusting the ranking.

Replay

Compare with known incidents using point-in-time inputs.

Inspect

Record missing paths, false leads and wrong assumptions.

Observe

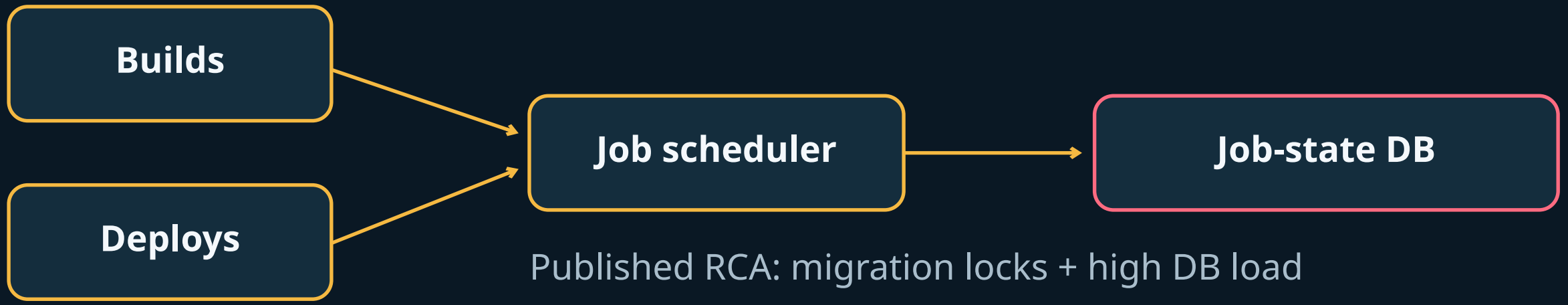
Run in shadow mode before relying on the output.

A repeatable calculation is not yet a validated model.

Shared dependency. Selective impact.

RENDER / OCT 14, 2026

Reconstructed dependencies / retrospective check, not a blind forecast



Published RCA: migration locks + high DB load

Builds + deploys disrupted

Live services unaffected

Research results: quality, warning time and noise.

Approach	F1	Lead time	FPR
Threshold baseline	0.52	3 min	18.0%
LSTM baseline	0.71	12 min	10.0%
DG-CFRM	**0.83**	**27 min**	**6.2%**

Paper Table III | one provider | reported results, not Render or an engineer trial.

Same framework. Different failure paths.

CLOUD PLATFORMS

Separate control-plane and live-serving dependencies.

A deploy outage need not mean an application outage.

EVENT-DRIVEN SYSTEMS

Model lag, retries and backlog recovery.

Success now can hide delayed work.

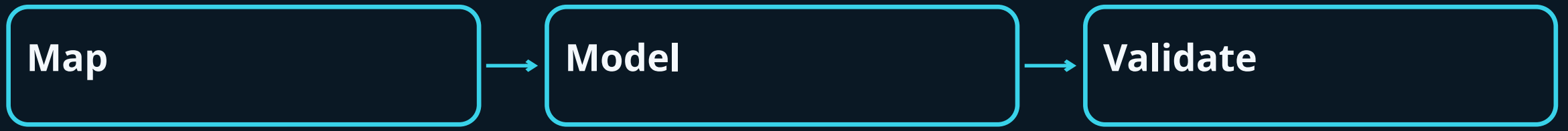
KUBERNETES

Track stable workloads and shared DNS, nodes and storage.

Replicas are not independent when infrastructure is shared.

Application guidance, not a claim of a measured Kubernetes deployment.

Try it on one journey next week.



Pick one critical user journey.

Test three failure scenarios.

Replay incidents. Check what improved.

Start small. Measure usefulness before building a platform.

Sources and further reading

RESEARCH PAPER

Joshi & Segireddy | Dependency Graph-Based Counterfactual Risk Estimation

doi.org/10.1109/TQCEBT67648.2026.11681845

AWS | OCTOBER 2025

DynamoDB / DNS disruption | official AWS post-event summary

aws.amazon.com/message/101925/

CLOUDFLARE | NOVEMBER 2025

Official analysis: failure from propagated bot-management feature file

blog.cloudflare.com/18-november-2025-outage/

RENDER | OCTOBER 2026

Incident history / scheduler migration postmortem

status.render.com/history

Research figures are reported results; the checkout simulations are illustrative, not measured production forecasts.

DEVFEST KC / 2026

Explore the research. Stay connected.

DevFest KC 2026 | Predictive Modeling of Cloud System Failures

Portfolio



akhiljoshi-portfolio.pages.dev

Google Scholar



Research & publications

LinkedIn



Connect professionally

Thank you to GDG Kansas City and everyone who joined the session.